

Customer Segmentation & Sales Performance Analysis

UK Online Retailer · 2009–2011

SQL · Python · Power BI

Author Md. Jubayer
Dataset Online Retail II (UCI) – Kaggle
Tools Python (Pandas), PostgreSQL, Power BI (DAX)
GitHub github.com/jubayer-cse/retail-analytics-project
Period December 2009 – December 2011
Rows 1,067,371 raw → 779,425 cleaned

Key Finding: The top 14% of customers (Champions segment, 839 customers) generated £9.88M — representing **56.9% of total revenue** of £17.37M. Combined with Loyal Customers, the top 35% of customers drove 76.7% of all revenue.

Contents

1	Project Overview	3
1.1	Background and Business Problem	3
1.2	Project Pipeline	3
1.3	Repository Structure	3
2	Dataset	4
2.1	Source and Description	4
2.2	Column Reference	5
3	Phase 1 — Data Cleaning (Python/Pandas)	5
3.1	Data Quality Issues Identified	5
3.2	Cleaning Code	6
3.3	Loading to PostgreSQL	7
4	Phase 2 — SQL Analysis (PostgreSQL)	7
4.1	Query 1: Monthly Revenue Trend	7
4.2	Query 2: Top 10 Products by Revenue	8
4.3	Query 3: Revenue by Country	8
4.4	Query 4: Repeat vs One-Time Customers	8
4.5	Query 5: Average Order Value by Month	9
5	Phase 3 — RFM Customer Segmentation	9
5.1	What is RFM?	9
5.2	RFM Calculation Code	10
5.3	Segment Results	11
6	Phase 4 — Power BI Dashboard	11
6.1	Data Model	11
6.2	DAX Measures	11
6.3	Calculated Columns	12
6.4	Dashboard Pages	13
6.5	Dashboard Features	13
7	Key Findings	13
7.1	Finding 1 — Revenue Concentration in Top Customer Segments	13
7.2	Finding 2 — Strong Q4 Seasonality	14
7.3	Finding 3 — Geographic Concentration Risk	14
7.4	Finding 4 — B2B Wholesale Buying Pattern	14
7.5	Finding 5 — High-Revenue and High-Volume Products Do Not Overlap	14
8	Business Recommendations	14
8.1	Recommendation 1 — Protect Champions: Retention Over Acquisition	14
8.2	Recommendation 2 — Q4 Planning: Build Inventory in September	15
8.3	Recommendation 3 — Re-engage At Risk Customers Immediately	15

8.4	Recommendation 4 — Eliminate Weekend Marketing Spend	15
8.5	Recommendation 5 — Evaluate International Expansion	15
9	Conclusion	15

1. Project Overview

1.1 Background and Business Problem

This report documents a complete end-to-end data analytics project analyzing real transactional data from a UK-based non-store online retailer selling unique all-occasion giftware. The dataset spans two full years (December 2009 to December 2011) and covers purchases from customers across 38+ countries.

The retailer's management required analytical answers to three core business questions:

1. **Customer value:** Which customers drive the most revenue, and is the business at risk of losing them?
2. **Product performance:** Which products should be prioritized for inventory, pricing, and marketing investment?
3. **Temporal and geographic patterns:** When and where does revenue concentrate, and how should the business plan around it?

1.2 Project Pipeline

The project follows a four-phase analytical pipeline, each phase producing outputs that feed the next:

Table 1: End-to-end project pipeline

Phase	Tool	Input	Output
1. Clean	Python / Pandas	Raw CSV (1.06M rows)	cleaned_retail_data.csv
2. Store	PostgreSQL	Cleaned CSV	transactions table
3. Query	SQL	transactions table	Business answers + CSVs
4. Segment	Python / RFM	Cleaned CSV	rfm_segments.csv
5. Visualize	Power BI / DAX	Both CSVs	4-page dashboard
6. Recommend	Written report	All findings	This document

How SQL and Power BI connect: Both tools read from the same `cleaned_retail_data.csv` independently. SQL demonstrates precise analytical querying. Power BI demonstrates visualization and DAX calculation. The SQL query result CSVs are stored in the GitHub repository as evidence of SQL work, not as data sources for Power BI.

1.3 Repository Structure

```

retail-analytics-project/
  notebooks/
    01_data_cleaning.ipynb
    02_load_to_sql.ipynb
    03_rfm_segmentation.ipynb
  sql/
    q1_monthly_revenue.sql
    q2_top_products.sql
    q3_revenue_by_country.sql
    q4_repeat_customers.sql
    q5_avg_order_value.sql
  powerbi/
    retail_dashboard.pbix
  data/cleaned/
    cleaned_retail_data.csv
    rfm_segments.csv
  reports/
    findings_and_recommendations.pdf
  images/
    dashboard_screenshots (4 pages)

```

2. Dataset

2.1 Source and Description

Table 2: Dataset properties

Property	Value
Name	Online Retail II (UCI)
Source	Kaggle — kaggle.com/datasets/mashlyn/online-retail-ii-uci
Rows	1,067,371 (before cleaning)
Columns	8
Time period	01/12/2009 to 09/12/2011 (two full years)
Countries	38+
File format	CSV (online_retail_II.csv, approx. 95MB)
Customer type	Mixed — individual consumers and wholesale buyers

2.2 Column Reference

Table 3: Column descriptions and rename actions

Original name	Renamed to	Description
Invoice	InvoiceNo	Transaction ID. Values starting with ‘C’ indicate cancellations
StockCode	(unchanged)	Unique product/item code
Description	(unchanged)	Product name
Quantity	(unchanged)	Units per line item (negative values = returns)
InvoiceDate	(unchanged)	Date and time of transaction
Price	UnitPrice	Price per unit in GBP (£)
Customer ID	CustomerID	Unique customer identifier (23% missing)
Country	(unchanged)	Customer’s country

Why three columns were renamed: The original file uses `Invoice` (not `InvoiceNo`), `Price` (not `UnitPrice`), and `Customer ID` with a space. Renaming immediately after loading ensures all downstream SQL queries, RFM code, and Power BI DAX formulas use consistent column names without throwing `KeyError` or `column-not-found` exceptions.

3. Phase 1 — Data Cleaning (Python/Pandas)

3.1 Data Quality Issues Identified

Table 4: Data quality issues and handling decisions

Issue	Rows affected	Action	Reason
Missing CustomerID	~243,000 (23%)	Dropped (dropna)	Cannot segment unknown customers
Cancelled orders	~9,288	Filtered out	Not completed sales
Negative quantity	Several thousand	Filtered (> 0)	Returns distort revenue
Zero/negative price	Several hundred	Filtered (> 0)	Data entry errors
Exact duplicates	Some	drop_duplicates	Avoid double-counting

Final result after cleaning: 779,425 rows — a reduction of 287,946 rows (27%) from the original dataset.

3.2 Cleaning Code

Listing 1: Complete data cleaning pipeline

```
import pandas as pd

# Load with ISO-8859-1 encoding (required for special
# characters
# in product descriptions exported from Excel)
df = pd.read_csv('online_retail_II.csv', encoding='ISO-8859-1',
)

# Rename columns to standard names used throughout the project
df = df.rename(columns={
    'Invoice': 'InvoiceNo',
    'Price': 'UnitPrice',
    'Customer ID': 'CustomerID'
})

# Step 1: Remove rows with missing CustomerID
# Cannot assign a transaction to a customer without an ID
# imputation is not possible for a unique identifier
df = df.dropna(subset=['CustomerID'])

# Step 2: Remove cancelled orders (InvoiceNo starting with 'C')
df = df[~df['InvoiceNo'].astype(str).str.startswith('C')]

# Step 3: Remove non-positive quantities and prices
# Negative quantity = return; zero/negative price = data error
df = df[(df['Quantity'] > 0) & (df['UnitPrice'] > 0)]

# Step 4: Remove exact duplicate rows
df = df.drop_duplicates()

# Step 5: Create TotalPrice column (used in all revenue
# calculations)
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

# Step 6: Convert InvoiceDate to datetime
# Required for date operations: extracting month, calculating
# recency days, sorting chronologically
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Sanity check
print(f"Final shape: {df.shape}")
print(df.isnull().sum())

# Export cleaned data
df.to_csv('cleaned_retail_data.csv', index=False)
```

3.3 Loading to PostgreSQL

Listing 2: Loading cleaned data into PostgreSQL

```
from sqlalchemy import create_engine
import pandas as pd

df = pd.read_csv('cleaned_retail_data.csv')

engine = create_engine(
    'postgresql://postgres:PASSWORD@localhost:5432/retail_analytics'
)

# method='multi' and chunksize dramatically improve
# performance
# for large datasets (default method inserts row-by-row which
# can take 30-60+ minutes for 779K rows)
df.to_sql(
    'transactions',
    engine,
    if_exists='replace',
    index=False,
    method='multi',
    chunksize=5000
)
print("Done      table loaded.")
```

4. Phase 2 — SQL Analysis (PostgreSQL)

4.1 Query 1: Monthly Revenue Trend

Business question: Is revenue growing? When does it peak seasonally?

Listing 3: Monthly revenue trend

```
SELECT
    DATE_TRUNC('month', "InvoiceDate"::timestamp) AS month,
    ROUND(SUM("TotalPrice")::numeric, 2) AS revenue
FROM transactions
GROUP BY month
ORDER BY month;
```

Note on ::timestamp cast: When a CSV is loaded into PostgreSQL via `to_sql()`, date columns are sometimes stored as text (`varchar`) rather than proper timestamp types. `DATE_TRUNC` requires a timestamp input, so the cast is required to avoid a function `date_trunc(unknown, text)` does not exist error.

4.2 Query 2: Top 10 Products by Revenue

Business question: Which products should be prioritized for inventory and marketing?

Listing 4: Top 10 products by revenue

```
SELECT
  "Description",
  SUM("Quantity") AS units_sold,
  ROUND(SUM("TotalPrice")::numeric, 2) AS revenue
FROM transactions
GROUP BY "Description"
ORDER BY revenue DESC
LIMIT 10;
```

4.3 Query 3: Revenue by Country

Business question: Where is revenue geographically concentrated?

Listing 5: Revenue by country

```
SELECT
  "Country",
  ROUND(SUM("TotalPrice")::numeric, 2) AS revenue,
  COUNT(DISTINCT "CustomerID") AS customers
FROM transactions
GROUP BY "Country"
ORDER BY revenue DESC;
```

Why COUNT(DISTINCT CustomerID): Plain COUNT(CustomerID) would count every transaction row — a customer who placed 50 orders would be counted as 50. DISTINCT ensures each customer is counted exactly once regardless of how many orders they placed.

4.4 Query 4: Repeat vs One-Time Customers

Business question: What share of customers ever returned for a second purchase?

Listing 6: Repeat vs one-time customers using a subquery

```
SELECT
  CASE WHEN order_count = 1 THEN 'One-time'
        ELSE 'Repeat'
  END AS customer_type,
  COUNT(*) AS num_customers
FROM (
  SELECT "CustomerID",
         COUNT(DISTINCT "InvoiceNo") AS order_count
  FROM transactions
  GROUP BY "CustomerID"
) sub
GROUP BY customer_type;
```

The inner subquery calculates how many distinct orders each customer placed. The outer query then labels customers as *One-time* (single order) or *Repeat* (two or more orders) and counts how many fall into each group.

4.5 Query 5: Average Order Value by Month

Business question: How much does a customer spend per complete order visit, and is it changing over time?

Listing 7: Average order value by month using nested subquery

```
SELECT
  DATE_TRUNC('month', "InvoiceDate"::timestamp) AS month,
  ROUND(AVG("order_total")::numeric, 2) AS avg_order_value
FROM (
  SELECT "InvoiceNo", "InvoiceDate",
         SUM("TotalPrice") AS order_total
  FROM transactions
  GROUP BY "InvoiceNo", "InvoiceDate"
) order_totals
GROUP BY month
ORDER BY month;
```

Why the subquery is essential: The transactions table has one row per product line item, not one row per order. Applying `AVG(TotalPrice)` directly would average individual product line values (e.g. £10, £8, £2) giving £6.67, when the customer's actual order total was £20. The inner subquery first collapses all line items from the same invoice into one true order total. The outer query then averages those real order totals by month.

5. Phase 3 — RFM Customer Segmentation

5.1 What is RFM?

RFM (Recency, Frequency, Monetary) is an industry-standard customer segmentation technique used by retail and e-commerce analytics teams worldwide. It groups customers based on three behavioral dimensions:

Table 5: RFM dimensions

Dimension	Question it answers	Business logic
Recency (R)	How recently did this customer buy?	Recent buyers are more likely to buy again
Frequency (F)	How many times have they bought?	Frequent buyers are more loyal and engaged
Monetary (M)	How much have they spent in total?	High spenders drive disproportionate revenue

5.2 RFM Calculation Code

Listing 8: RFM score calculation

```
import pandas as pd
from datetime import timedelta

df = pd.read_csv('cleaned_retail_data.csv')
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Snapshot date = one day after the last transaction
# This is the reference point for recency calculation
snapshot_date = df['InvoiceDate'].max() + timedelta(days=1)

# Calculate R, F, M per customer
rfm = df.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalPrice': 'sum'
})
rfm.columns = ['Recency', 'Frequency', 'Monetary']

# Score each metric 1-4 using quartiles
# Recency: lower days = more recent = better => labels
# reversed [4,3,2,1]
rfm['R_score'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1])
rfm['F_score'] = pd.qcut(
    rfm['Frequency'].rank(method='first'), 4, labels=[1,2,3,4]
)
rfm['M_score'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4])

# Combine into 3-digit score string (e.g. '444' = best
# customer)
rfm['RFM_Score'] = (rfm['R_score'].astype(str)
                    + rfm['F_score'].astype(str)
                    + rfm['M_score'].astype(str))

# Assign segments based on score combinations
def segment(row):
    if row['RFM_Score'] in ['444', '434', '443']:
        return 'Champions'
    elif row['R_score'] in [3, 4] and row['F_score'] in [3, 4]:
        return 'Loyal Customers'
    elif row['R_score'] in [1, 2] and row['F_score'] in [3, 4]:
        return 'At Risk'
    elif row['R_score'] == 1 and row['F_score'] == 1:
        return 'Lost'
    else:
        return 'Potential Loyalist'
```

```
rfm['Segment'] = rfm.apply(segment, axis=1)
rfm.to_csv('rfm_segments.csv')
```

5.3 Segment Results

Table 6: RFM segment distribution and revenue contribution

Segment	Customers	% of base	Revenue (£)	% of revenue	AOV (£)
Champions	839	14.3%	9,883,516	56.9%	564
Loyal Customers	1,220	20.8%	3,446,019	19.8%	379
At Risk	880	15.0%	2,165,028	12.5%	377
Potential Loyalist	2,167	36.9%	1,626,686	9.4%	422
Lost	772	13.1%	253,556	1.5%	328
Total	5,878	100%	17,374,804	100%	470

Key insight: Champions (14.3% of customers) generate 56.9% of revenue. Combined with Loyal Customers, the top 35% of customers drive 76.7% of total revenue. In contrast, Lost customers (13.1% of the base) generate only 1.5% of revenue despite a similar headcount to Champions — demonstrating that customer value distribution is highly skewed.

6. Phase 4 — Power BI Dashboard

6.1 Data Model

Two tables are imported into Power BI and connected via a relationship on `CustomerID`:

Table 7: Power BI data model

Table	Source	Rows	Relationship side
cleaned_retail_data	.csv	779,425	Many (many transactions per customer)
rfm_segments	.csv	5,878	One (one segment per customer)

6.2 DAX Measures

All KPIs in the dashboard are calculated using DAX measures that respond dynamically to filter context:

Listing 9: Core DAX measures

```
-- Basic KPIs
Total Revenue = SUM(cleaned_retail_data[TotalPrice])
Total Orders = DISTINCTCOUNT(cleaned_retail_data[InvoiceNo])
```

```

Total Customers = DISTINCTCOUNT(cleaned_retail_data[CustomerID
])
Average Order Value = DIVIDE([Total Revenue], [Total Orders],
0)
Total Units Sold = SUM(cleaned_retail_data[Quantity])

-- Repeat customer measures
Repeat Customers =
CALCULATE(
    DISTINCTCOUNT(cleaned_retail_data[CustomerID]),
    FILTER(
        VALUES(cleaned_retail_data[CustomerID]),
        CALCULATE(DISTINCTCOUNT(cleaned_retail_data[InvoiceNo
]))) > 1
    )
)
Repeat Customer Rate = DIVIDE([Repeat Customers], [Total
Customers], 0)

-- Segment measures
Champions Revenue % =
DIVIDE(
    CALCULATE([Total Revenue], rfm_segments[Segment] = "
Champions"),
    [Total Revenue], 0
)

Segment Revenue % of Total =
DIVIDE(
    [Total Revenue],
    CALCULATE([Total Revenue], ALL(rfm_segments[Segment])),
    0
)

```

6.3 Calculated Columns

Listing 10: DAX calculated columns

```

-- Day of week for sorting and heatmap rows
DayOfWeek = FORMAT(cleaned_retail_data[InvoiceDate], "dddd")
DayNumber = WEEKDAY(cleaned_retail_data[InvoiceDate], 2)

-- Hour of day for time-of-day analysis
HourOfDay = HOUR(cleaned_retail_data[InvoiceDate])

-- Year and quarter for trend comparisons
Year = YEAR(cleaned_retail_data[InvoiceDate])
Quarter = "Q" & FORMAT(cleaned_retail_data[InvoiceDate], "Q")

```

6.4 Dashboard Pages

Table 8: Power BI dashboard pages and visuals

Page	Key visuals and features
Executive Overview	5 KPI cards (Revenue, Orders, AOV, Customers, Repeat Rate), monthly revenue line chart with average line, revenue by country bar chart (Top 10), year-over-year comparison, day-of-week chart, year slicer
Product Performance	Top 10 products by revenue (horizontal bar), top 10 products by units sold (horizontal bar), product detail table with StockCode and AOV, date range slicer, country dropdown slicer
Customer Segmentation	Three KPI cards (Champions Revenue %, At Risk Count, Lost Count), customer count donut chart, revenue by segment bar chart, Frequency vs Monetary scatter plot coloured by segment, segment summary table, tile-style segment slicer
Trends & Seasonality	Monthly revenue line with average line, revenue by hour-of-day column chart, day-of-week vs hour-of-day heatmap matrix with conditional formatting, quarterly YoY comparison coloured by year, year tile slicer, month dropdown slicer

6.5 Dashboard Features

- **Navigation buttons:** Header bar on every page with four buttons providing one-click navigation between pages. The current page button is highlighted in accent blue; all others are unstyled.
- **Reset Filters bookmarks:** Each page has a Reset Filters button linked to a bookmark that captures all slicers in their default (All) state, providing one-click filter reset.
- **Custom tooltips:** Key visuals have additional fields added to the Tooltips field well, so hovering over any data point shows contextual information beyond the primary value.
- **Cross-filtering:** All slicers filter every visual on the page simultaneously, enabling interactive exploration.
- **Conditional formatting heatmap:** The day-of-week vs hour-of-day matrix on Page 4 uses background colour scaling to show order concentration visually.

7. Key Findings

7.1 Finding 1 — Revenue Concentration in Top Customer Segments

Champions (839 customers, 14.3% of the base) generated £9.88M — 56.9% of total revenue. Combined with Loyal Customers, the top 35% of customers drove 76.7% of total revenue (£13.33M of £17.37M). Lost customers (13.1% of the base) generated only £253,556 — 1.5% of revenue — despite a similar customer count to Champions.

7.2 Finding 2 — Strong Q4 Seasonality

Revenue peaks sharply in Q4 across both full years (2010 and 2011). November reached £2.07M and December £2.32M — more than double the H1 monthly average of approximately £1.1M. This pattern is consistent year-over-year, confirming it is a structural seasonal pattern rather than a one-off event.

7.3 Finding 3 — Geographic Concentration Risk

The United Kingdom generated £14.4M — approximately 83% of total revenue. EIRE and Netherlands are the next largest markets at approximately £0.6M each (under 4% each). Revenue is highly concentrated in the home market.

7.4 Finding 4 — B2B Wholesale Buying Pattern

72.39% of customers placed more than one order — a strong repeat rate. Peak ordering occurs between 10am–12pm on weekdays, with Thursday being the single busiest day. Saturday generated only 30 total orders across the entire two-year dataset. Average order value of £469.98 is far above typical consumer retail, confirming a predominantly B2B wholesale customer base rather than individual consumers.

7.5 Finding 5 — High-Revenue and High-Volume Products Do Not Overlap

REGENCY CAKESTAND 3 TIER is the top revenue product (£277,656, 24,124 units) but does not appear in the top 10 by volume. WORLD WAR 2 GLIDERS ASSTD DESIGNS sells the most units (~100K) but does not appear in the top 10 by revenue — a high-volume, low-margin item. WHITE HANGING HEART T-LIGHT HOLDER appears in both top 10 lists, making it the most strategically important product overall.

8. Business Recommendations

8.1 Recommendation 1 — Protect Champions: Retention Over Acquisition

Losing 10% of the Champions segment would remove approximately £988,000 from annual revenue. The business should implement a dedicated loyalty program, early product access, or account management for this group *before* increasing acquisition marketing spend. Champions have an average order value of £564 — significantly above the overall average of £470 — making each Champions customer individually more valuable than any other segment.

8.2 Recommendation 2 — Q4 Planning: Build Inventory in September

The consistent Q4 revenue surge requires proactive preparation. Inventory buildup should begin in September to prevent stockouts during November–December peak demand. Marketing campaigns should launch in October to capture early Q4 buyers. Staffing and customer service capacity should be increased in Q4. Major operational changes or system upgrades should be avoided during Q4 when order volume is at its highest.

8.3 Recommendation 3 — Re-engage At Risk Customers Immediately

880 customers previously bought frequently but have not purchased recently. These customers are at high risk of transitioning into the Lost segment. A targeted win-back email campaign for the At Risk segment is significantly more cost-effective than new customer acquisition, as these customers already have demonstrated purchase intent and product familiarity.

8.4 Recommendation 4 — Eliminate Weekend Marketing Spend

Saturday generated only 30 total orders across the entire two-year dataset. The customer base is predominantly wholesale B2B buyers who place orders during working hours. Weekend marketing campaigns are almost entirely wasted budget. Redistributing this spend to Thursday morning campaigns (the single busiest day and time) would yield meaningfully higher return on investment.

8.5 Recommendation 5 — Evaluate International Expansion

EIRE and Netherlands each generate approximately £0.6M in revenue despite likely receiving zero targeted marketing investment. This suggests latent demand in these markets that could be activated with modest localization effort (local currency display, regional promotional campaigns). Geographic concentration at 83% UK revenue also represents a business risk if UK economic conditions deteriorate.

9. Conclusion

This project demonstrates a complete, production-realistic data analytics workflow applied to real, messy transactional data. The four-phase pipeline — cleaning in Python, querying in SQL, segmenting with RFM, and visualizing in Power BI — covers the exact skill set required for a professional Data Analyst role.

The central finding — that 14% of customers generate 57% of revenue — is not a constructed result but an empirical one derived from 1 million+ real transactions. This concentration finding has direct, specific business implications that are articulated in the recommendations above.

The dashboard provides a self-serve analytical tool that enables non-technical stakeholders to explore revenue patterns, customer segments, and product performance interactively without requiring analyst intervention for each query. This is the primary deliverable of a working data analyst: not a report, but a repeatable, accessible analytical capability.

Table 9: Project deliverables summary

Deliverable	Description
01_data_cleaning.ipynb	Python cleaning pipeline with documented decisions
02_load_to_sql.ipynb	PostgreSQL loading with performance optimization
03_rfm_segmentation.ipynb	Complete RFM model with segment assignment logic
5 SQL query files	Business queries with comments explaining each decision
retail_dashboard.pbix	4-page interactive Power BI dashboard
rfm_segments.csv	5,878-row segmentation output
This report	Written findings and business recommendations

Md. Jubayer · Data Analyst · github.com/jubayer-cse/retail-analytics-project